

Seminar 4
Object-Oriented Design, IV1350

Ellen Grönholm
ellengro@kth.se

May 17, 2026

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

Usage of AI

The usage of AI assistance in the report is limited to help of environmental issues with the IDE and explaining observed errors when coding. As AI assistance ChatGPT-5.5 (OpenAI) and Gemini 3 Flash (Google) have been used.

1 Introduction

This report describes the enhancements made to the Electric Bike Repair Workshop system for Seminar 4. The primary focus is the implementation of robust error handling through Java exceptions and the application of Design Patterns to improve system decoupling and extensibility. Specifically, this iteration addresses alternative flows involving missing data and database failures, while utilizing the Observer, Singleton, and Strategy patterns to manage system state and business logic.

2 Method

2.1 Exception Handling Strategy

Although it was not needed, to get a feel of where to start regarding the exception handling, the UML was used. In the communication diagrams the exceptions were added for the flows where it was deemed necessary. Starting from this, the exceptions were first created, then at the lowest applicable layer thrown. Then the methods are followed upwards to catch the thrown exceptions.

When implementing exceptions, multiple factors were considered. All exceptions thrown, had to be logged in a file for the developers to see, according to the instructions, so all errors will create an unchecked output. Most exceptions also throw a checked exception containing less information so that the user can respond on the exception. Mostly the checked exceptions tell the user to try again while returning the entered data that was not applicable.

When creating the exceptions, it has been decided not to make them specific. They are thus applicable for multiple databases or objects. The reason this abstraction level is chosen is as this better decouples as well as promotes low coupling and high cohesion. While it is still deemed that the exception can be understood from the more general name instead of having a specific name. *CustomerNotFoundException* was thus changed to *ObjectNotFoundException* so that the exception was applicable for both the customer and the repair order. This also makes it easier to scale the code with more objects that might not be found. A development would be to allow the **FaultList** and **ProposedRepairList** to throw the exception. However, the names were decided to be kept so specific that the error would be understandable only from the name.

Lastly, logging was used to be able to work and better the code based on the thrown errors. The logs are more detailed for developers and saved in txt-files with the name *logDateOfError*.

After creating all exceptions, tests for these exceptions were added.

2.2 Design Patterns Selection

Patterns help clarify the code and better coupling and cohesion. That an Observer pattern was implemented was due to the instructions enforcing it. To meet the requirements, the Strategy and Singleton patterns were also implemented:

2.2.1 Observer Pattern

The Observer pattern was made to notify the receptionist and the mechanic when changes were done. Before implementing the Observer pattern, the code already had a `print` statement to notify amount this. It was thus simple to implement an observer interface with a method `notify` which did this.

By implementing this, however, coupling was held lower as the `RepairOrder` class did not have to know about both all observers (View and Logger). Without the Observer pattern, the `RepairOrder` class would need a direct reference to the `RepairOrderView` and the `RepairOrderLogger` to update the screen. This would create a 'hard dependency'. All in all, the Observer pattern keeps UI logic out of business classes. This also helps with handling complexity without if-else statements.

When implementing this it had to be decided what to send through the 'notify' function from the `repairOrder` to the observers. First the thought was to send a `RepairOrderDTO` through. This DTO however contained information, but it was hard to output it to the view. Therefor, it was then considered converting the repair order to a string and then passing a String through the `notify` function. The problem with this was the lack of flexibility at the observers end. The code to print the the repair order is long, so it was not desirable to rewrite all that code for the `RepairOrderDTO`. The solution was to make a `RepairOrderInterface` that is implemented by both the `RepairOrder` and the `RepairOrderDTO`. The print function was thus set to take objects of 'RepairOrderInterface', thus making the code usable in all cases.

2.2.2 Singleton Pattern (GoF)

The Singleton pattern was used at the instructions urging. It was specifically used for the `CustomerRegistry` to ensure only one instance of the database interfaces exists. The implementation followed by adding a static variable; `CustomerRegistrySingleton INSTANCE = new CustomerRegistrySingleton()`. This variable can not be accessed directly, and is instead reached through `getInstance()`.

The biggest hardship regarding the Singleton pattern is that the testing had to be changed and most of the earlier codes `@Disable` as the Customer registry could not be reset for each test.

2.2.3 Strategy Pattern (GoF)

At the instructions appeal, the Strategy Pattern was also added. This was implemented to handle different discount pricing and allows the workshop to change discount algorithms at runtime without modifying the `RepairOrder` class. For this a new layer was implemented; the `payment` layer, in which a payment Strategy interface was added and the different strategies. To this, a new class `PaymentProcessor` was created to take care of payment structures. The thought is that this class can be expanded to handle more than only discounts as time goes by.

At this time, only discounting for the total price has been implemented, but it is simple to create a method in the `PropossedRepairs` class that also sends the individual prices via the `PaymentProcessor` class so to change prices of those.

As of now the discount is not printed on the receipt either. Instead the discount is applied on the total directly.

3 Result

3.1 Implementation of Exceptions

No view has been made, so in the view file the methods are called to the controller which would be done via a nicer view in other cases. The output thus also is based on how the view is programmed at the moment. If the basic flow is followed Figure 1 is the output.

```
The program has started!
-----

Created Repair Order ID: 0
Problem Description completed.
Diagnostic report started.
Diagnostic report completed.
Proposed repairs completed.
order set to CUSTOMER_REVIEW

Customer Info:
-----
Name:          ellen
Phone Number:  0733976992
Email Address:  ellen.gronholm@hotmail.com
Bike:
    SerialNumber: 23490
    Model:        dam
    Brand:        helkama

Problem Description:
-----
Current bike faults:
- TIRE
- BRAKES

Diagnostic Report:
-----
Current bike faults:
- TIRE
- BRAKES

Propossed Repairs:
-----
Current recommended bike repairs:
- Replace tire:      200
- Fix chain:        150
  total:             350

Date:              Sat May 16 23:08:09 CEST 2026
ID:                0

Customer accepts the proposed repairs
.

receiptRepairOrder0 U X
Uni2025 > iv1350objectorienteraddressign > bikerepairshop > ↓ receipt
1  Customer Info:
2  | -----
3  Name:          ellen
4  Phone Number:  0733976992
5  Email Address:  ellen.gronholm@hotmail.com
6  Bike:
7  |   SerialNumber: 23490
8  |   Model:        dam
9  |   Brand:        helkama
10 |
11 Problem Description:
12 | -----
13 Current bike faults:
14 | - TIRE
15 | - BRAKES
16 |
17
18 Diagnostic Report:
19 | -----
20 Current bike faults:
21 | - TIRE
22 | - BRAKES
23 |
24
25 Propossed Repairs:
26 | -----
27 Current recommended bike repairs:
28 | - Replace tire:      200
29 | - Fix chain:        150
30 |   total:             350
31 |
32 Date:              Sat May 16 23:08:09 CEST 2026
33
34 ID:                0
```

Figure 1: The output from the basic flow and the printable file created as the receipt.

Since the view is not made, all commands are hard coded. It is however written as if the input could have come from the user. The code thus uses while loops that break only when conditions are met. Thus, when running the alternative flow 5a), the program doesn't wait for new inputs, it runs the hard coded wrong answer, generating the output in Figure 2 a). In Figure 2 b) the output for the error when the database cannot be establish connection is shown.

```

The program has started!
-----

The Customer could not be found.
The data you entered was: 0733976993
The Customer could not be found.
The data you entered was: 0733976993
The Customer could not be found.
The data you entered was: 0733976993
The Customer could not be found.
The data you entered was: 0733976993

```

(a)

```

The program has started!
-----

In the Customer Registry the following error occurred: Connection to the Database was never established

```

(b)

Figure 2: a) The output of running the alternative flow 5a with the wrong answer hard-coded. b) The output if the database is never connected to the program.

Since these alternative flows throw exceptions, the developers must be able to see these exceptions. The exceptions are thus logged in txt-files. These logs are shown in Figure 3.

```

log136-23.25.55.txt X
Uni2025 > iv1350objectorienteraddesign > bikerepairshop > log136-23.25.55.txt
1 The object Customer could not be found for the phone number 0733976993.
2 Error message: The phone number was not found in the database.
3
4 16 maj 2026 23:25:55
5

```

(a)

```

log136-23.40.22.txt X
Uni2025 > iv1350objectorienteraddesign > bikerepairshop > log136-23.40.22.txt
1 The Customer Registry Connection to the Database was never established
2
3 16 maj 2026 23:40:22
4
5

```

(b)

Figure 3: a) The logged exception for the alternative flow 5a. b) The logged exception for failure to establish connection with databases.

3.2 Observer Pattern Implementation

Since the view is not created, meaning that all instructions are hard coded and all outputs flow in the same terminal, the output when adding the notifications is a bit jumbled. To simplify the reading of the output, a yellow and an orange box have been drawn over Figure 4. The yellow box signifies the notification message to the mechanic, and the orange is the message to the receptionist.

```

The program has started!
-----
Created Repair Order ID: 0
Hi Mechanic, there is a new order to be reviewed:
Customer Info:
-----
Name: ellen
Phone Number: 0733976992
Email Address: ellen.gronholm@hotmail.com
Bike:
    SerialNumber: 23490
    Model: dam
    Brand: helkama

Problem Description:
-----
Current bike faults:
- TIRE
- BRAKES

Date: Sun May 17 14:51:48 CEST 2026
ID: 0

Problem Description completed: Current bike faults:
- TIRE
- BRAKES

```

```

Diagnostic report started.
Diagnostic report completed.
Hi, the order 0 has been fixed by the mechanic and is ready to be reviewed:
Customer Info:
-----
Name: ellen
Phone Number: 0733976992
Email Address: ellen.gronholm@hotmail.com
Bike:
    SerialNumber: 23490
    Model: dam
    Brand: helkama

Problem Description:
-----
Current bike faults:
- TIRE
- BRAKES

Diagnostic Report:
-----
Current bike faults:
- TIRE
- BRAKES

Proposed Repairs:
-----
Current recommended bike repairs:
- Replace tire: 200
- Fix chain: 150
    total: 350

Date: Sun May 17 14:51:48 CEST 2026
ID: 0

Proposed repairs completed.

```

```

order set to CUSTOMER_REVIEWING
Customer Info:
-----
Name: ellen
Phone Number: 0733976992
Email Address: ellen.gronholm@hotmail.com
Bike:
    SerialNumber: 23490
    Model: dam
    Brand: helkama

Problem Description:
-----
Current bike faults:
- TIRE
- BRAKES

Diagnostic Report:
-----
Current bike faults:
- TIRE
- BRAKES

Proposed Repairs:
-----
Current recommended bike repairs:
- Replace tire: 200
- Fix chain: 150
    total: 350

Date: Sun May 17 14:51:48 CEST 2026
ID: 0

Customer accepts the proposed repairs

```

Figure 4: The output from the basic flow and the notifications to the mechanic (yellow) and the receptionist(orange).

The notifications are simpler to see when logging the output instead, as seen in figure 5

```

CustomerReview137-15.25.57.txt M
-----
1 Hi Mechanic, there is a new order to be reviewed:
2 Customer Info:
3 -----
4 Name: ellen
5 Phone Number: 0733976992
6 Email Address: ellen.gronholm@hotmail.com
7 Bike:
8     SerialNumber: 23490
9     Model: dam
10    Brand: helkama
11
12 Problem Description:
13 -----
14 Current bike faults:
15 - TIRE
16 - BRAKES
17
18 Date: Sun May 17 15:25:57 CEST 2026
19
20 ID: 0
21
22 17 maj 2026 15:25:57
23
24
25

```

```

CustomerReview137-15.25.57.txt M
-----
1 Hi, the order 0 has been fixed by the mechanic and is ready to be reviewed:
2 Customer Info:
3 -----
4 Name: ellen
5 Phone Number: 0733976992
6 Email Address: ellen.gronholm@hotmail.com
7 Bike:
8     SerialNumber: 23490
9     Model: dam
10    Brand: helkama
11
12 Problem Description:
13 -----
14 Current bike faults:
15 - TIRE
16 - BRAKES
17
18 Diagnostic Report:
19 -----
20 Current bike faults:
21 - TIRE
22 - BRAKES
23
24 Proposed Repairs:
25 -----
26 Current recommended bike repairs:
27 - Replace tire: 200
28 - Fix chain: 150
29     total: 350
30
31 Date: Sun May 17 15:25:57 CEST 2026
32
33 ID: 0
34
35 17 maj 2026 15:25:57
36
37
38

```

Figure 5: The output from the basic flow and the printable file created as the receipt.

3.3 Additional GoF Patterns

3.3.1 Singleton: Registry Access

The Singleton implementation didn't change to output as it just ensured that multiple databases cannot be created. The code behind the Singleton was also explained in the Method section 2.2.2, so it will not be explained more here.

3.3.2 Strategy: Discount Logic

The Discount logic is very simple at this stage of the implementation, and multiple advances on this code has already been noted. This is discussed in the Method section 2.2.3. This new output is seen in Figure 6, where a 15% discount is applied to the total.

```
Propossed Repairs:
-----
Current recommended bike repairs:
- Replace tire:      200
- Fix chain:    150
                total:      298
```

Figure 6: The output from the basic flow and the printable file created as the receipt.

For the discount, it was chosen to always round the discounted price to the next whole number. This is because in countries like Sweden, cents/*öre* are out of use, and if rounding to the nearest number, then money is lost.

3.4 Git Repository

<https://gits-15.sys.kth.se/ellengro/IV1350-ObjektOrienteradDesign>

4 Discussion

4.1 Encapsulation and Cohesion

The use of exceptions improves encapsulation by preventing the View from needing to check return values (like `null` or `-1`) for error states. By sending errors instead of 'irrelevant' values up the layers, cohesion is kept higher as the information sent is always relevant. When a low-level error occurs for example, the data Layer catches the technical error and wraps it in a more general exception. This keeps the higher layers focused on their logic rather than technical troubleshooting. Coupling also stays low as higher levels do not need to understand the exception of failure, they only need to catch it.

The exception implementation also made it easier to implement the possibility to notify users and developers without creating code greatly based on if statements clouding the code. However, the down side is that a the code now contains a lot of catches as logging is needed for both developers and users, for all methods based on the data base.

4.2 Pattern Application and Quality

The main reason to implement the Observer pattern is the increased cohesion accomplished by moving logging and update logic out of the business classes. The Singleton pattern prevents data sync issues by ensure we don't have 100 copies of a Database Registry, and while this is not yet of greatest concern in our program, if another developer were to create a data-base. This code is good to have implemented. The Strategy pattern allows for simple addition and change of payment rules by separating this logic and allowing easy switching without a crazy if-else statement.

4.2.1 Encapsulation

By using Exception Abstraction the technical implementation was further encapsulated as the View doesn't need to know if an exception came from a SQL database or a CSV file. The "how" of the failure is hidden inside the exception object. The Strategy pattern works in a similar way, as classes don't need to know the math behind a discount. Instead they know to always call for a `DiscountStrategy`. The specific logic is encapsulated within the strategy classes.

4.2.2 Cohesion

Before exceptions, methods often returned 'fault' values (like -1 or null) to signal errors or that an object wasn't found. This forced the business logic to handle error checking. With exceptions, the code stays clean and focused only on the business task, while error handling is offloaded to catch blocks. By using the Strategy pattern, the code could be separated to small, highly cohesive classes, instead of one giant Payment class with 500 lines of different discount rules.

The Observer pattern significantly increased cohesion. The RepairOrder no longer carries the "extra baggage" of updating the screen or writing to a log file. It only manages repair data. The Logger and View classes handle the output, keeping all responsibilities separate.

4.2.3 Coupling

Generic exceptions reduce coupling between layers. If the Integration Layer throws a generic DatabaseException, the other layers do not need to know the extent of the error, instead the exception can propagate upwards until given to the right person to fix. Regarding the Strategy pattern, the RepairOrder is coupled to an Interface (DiscountStrategy) rather than a Concrete Class (Discount20). This means you can swap logic at runtime without the main class ever knowing.

As with cohesion, the Observer Pattern affected the coupling greatly. In a high-coupled system, RepairOrder would have a variable like RepairOrderView view. In the

new design, RepairOrder has no idea the View exists. It only knows it has a list of anonymous Observers. These observers can easily be added, removed, or changed without touching the Domain layer.

4.2.4 Encountered Problems

These changes were not flawless. The Observer pattern replaced the manual view, but only partly, thus creating integration problems between the observers and the originally output flow. The implementation of the singleton made testing harder by introducing global state issues. There is also a risk with the Strategy pattern as is now, that a strategy is applied, but then never removed, thus allowing prices to be wrong in the code.

4.3 Conclusion

The integration of Design Patterns and formal Exception Handling has transitioned the Electric Bike Workshop system from a basic prototype to a safer and more usable architecture that is resilient to errors and easy to extend.

5 References

- Geekific. (2021a, May 22). The Singleton pattern explained and implemented in Java | Creational Design Patterns | Geekific [Video]. YouTube.
<https://www.youtube.com/watch?v=tSZn4wkBIu8>
- Geekific. (2021b, September 25). The strategy pattern explained and implemented in Java | Behavioral Design Patterns | Geekific [Video]. YouTube.
<https://www.youtube.com/watch?v=Nrwj3gZiuJU>
- GeeksforGeeks. (n.d.). Java Interface. GeeksforGeeks.
<http://www.geeksforgeeks.org/java/interfaces-in-java/>
- GeeksforGeeks. (2025, July 11). Java current date and time. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/java-current-date-time/>
- GeeksforGeeks. (2026, March 13). File handling in Java. GeeksforGeeks.
<https://www.geeksforgeeks.org/java/file-handling-in-java/>
- Lindbäck, L. (March 23 2026) *A First Course in Object-Oriented Development: A Hands-On Approach*.
- Marić, P. (2024, January 8). The Observer Pattern in Java. Baeldung.
<https://www.baeldung.com/java-observer-pattern>

- Paraschiv, E. (2023, December 1). Java – Write to File. Baeldung.
<https://www.baeldung.com/java-write-to-file>